

ΠΑΡΑΡΤΗΜΑ Δ

Τεχνικός Οδηγός Υλοποίησης

*Ευφυής Πράκτορας Αναπλήρωσης Αποθεμάτων
με Διασύνδεση SAP*

Φοιτητής:	George Drakos
Επιβλέπων:	Σωτήρης Γκαγιαλής (ΕΜΠ Μηχανολόγων)
Πρόγραμμα:	ΔΔΠΜΣ Athens MBA (ΟΠΑ + ΕΜΠ)
GitHub:	github.com/greتل/replenishment-agent
Έκδοση Project:	v1.0 (μετά Bootstrap CI, Rule Ablation, Forecast)
Tests:	183/183 passing
LOC:	~7,500 Python + ABAP

Σκοπός Αυτού του Οδηγού

Το παρόν παράρτημα συνοδεύει τη Διπλωματική Εργασία και τεκμηριώνει λεπτομερώς την τεχνική υλοποίηση του ευφυούς πράκτορα αναπλήρωσης αποθεμάτων. Ο στόχος είναι ο αναγνώστης — είτε επιβλέπων είτε μέλος της τριμελούς επιτροπής — να μπορεί να κατανοήσει: (α) τι ακριβώς υλοποιήθηκε, (β) γιατί επιλέχθηκε αυτή η αρχιτεκτονική, (γ) πώς να αναπαράγει τα αποτελέσματα.

Το έγγραφο διαχωρίζεται σε εννέα ενότητες που ακολουθούν τη λογική ροή του project: από την αρχιτεκτονική, στα δεδομένα, στον αλγόριθμο, στα rules, στην αξιολόγηση, και τέλος στις προσθήκες που αναβαθμίζουν τη ΔΕ σε εξαιρετικού επιπέδου εργασία (Bootstrap Confidence Intervals, Rule Ablation Study, Demand Forecast Dashboard, AI Copilot).

Πίνακας Περιεχομένων

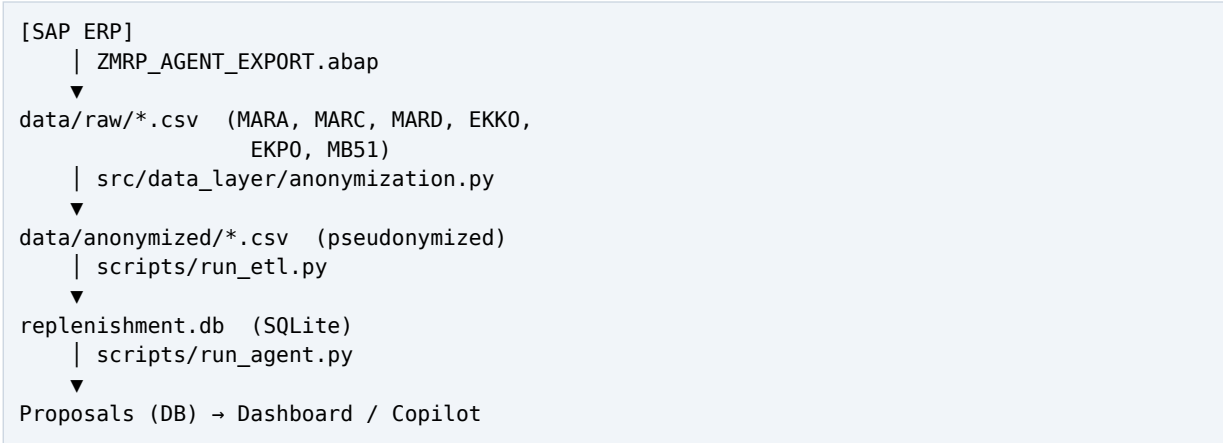
1. Αρχιτεκτονική Συστήματος
2. Επίπεδο Δεδομένων (Data Layer)
3. Πράκτορας BDI (Agent)
4. MRP & Lot Sizing
5. Επιχειρησιακοί Κανόνες (Rules)
6. Αξιολόγηση (Backtest) με Πολλαπλά Σενάρια
7. Νέα Προσθήκη: Bootstrap Confidence Intervals
8. Νέα Προσθήκη: Rule Ablation Study
9. Νέα Προσθήκη: Demand Forecast Dashboard
10. AI Copilot (Ollama-based)
11. Πώς να Τρέξετε τον Κώδικα — Βήμα προς Βήμα
12. Defense-Ready Σημεία Συζήτησης

1. Αρχιτεκτονική Συστήματος

Το σύστημα ακολουθεί τρι-επίπεδη αρχιτεκτονική (three-tier architecture), σχεδιασμένη για offline λειτουργία και απόλυτη ασφάλεια εταιρικών δεδομένων:

Επίπεδο	Τεχνολογία	Ρόλος
Παρουσίαση	Streamlit Dashboard	Visualization 6 tabs
Λογική	Python BDI Agent	Perception → Deliberation → Action
Δεδομένων	SQLite + SQLAlchemy ORM	Persistence, ETL από SAP CSV

1.1 Ροή Δεδομένων



1.2 Γιατί Αυτή η Αρχιτεκτονική;

- **Security & Privacy:** Τα εταιρικά δεδομένα παραμένουν στο local SQLite. Καμία απευθείας σύνδεση με production SAP system, ώστε να μη δημιουργείται φόρτος ή κίνδυνος.
- **Reproducibility:** Όλη η ροή είναι deterministic. Με ίδια δεδομένα και ίδιο seed, ο πράκτορας παράγει πάντα τις ίδιες προτάσεις. Κρίσιμο για ακαδημαϊκή έρευνα.
- **Modularity:** Κάθε επίπεδο είναι ανεξάρτητο. Μπορώ να αλλάξω το data layer (π.χ., από SQLite σε PostgreSQL) χωρίς να αγγίξω τον agent.
- **Testability:** Το data layer είναι in-memory στα tests (sqlite:///memory:), επιτρέποντας ταχύτατη εκτέλεση 183 unit tests σε ~17 δευτερόλεπτα.

1.3 Δομή Φακέλων



2. Επίπεδο Δεδομένων

2.1 Πηγαία Δεδομένα από SAP

Το ABAP πρόγραμμα ZMRP_AGENT_EXPORT εξάγει 6 πίνακες από το SAP σε μορφή CSV:

SAP Table	Περιεχόμενο	Χρήση στο Project
MARA	General material master	material_id, type, UoM, weight
MAKT	Material descriptions	Περιγραφή υλικού (multi-language)
MARC	Material per plant	MRP type, lead time, safety stock
MARD	Stock per storage location	Διαθέσιμο απόθεμα ανά location
EKKO	PO header	Vendor, currency, PO date
EKPO	PO line items	Quantity, expected delivery date
MB51 (MSEG+MKPF)	Material movements	Goods receipts & issues

Τεχνική σημείωση για το ABAP: Το extractor κάνει LEFT OUTER JOIN ανάμεσα σε MARA και MAKT για να φέρει τις περιγραφές των υλικών, με προτεραιότητα στη γλώσσα που επιλέγει ο χρήστης (παράμετρος p_lang) και fallback στα Αγγλικά. Το MB51 χρησιμοποιεί το πεδίο SHKZG (S=Soll/Debit για receipts, H=Haben/Credit για issues) για να καθορίσει το πρόσημο της ποσότητας — λάθος που συναντάται συχνά σε ad-hoc ABAP scripts.

2.2 Anonymization (GDPR/Privacy)

Το src/data_layer/anonymization.py μετατρέπει τα raw SAP exports σε ανωνυμοποιημένα CSV που μπορούν να χρησιμοποιηθούν για ακαδημαϊκούς σκοπούς:

Πεδίο	Αρχικό	Μετά
Material number	000000000000020084	MAT9103C8C8 (MD5-based)
Plant	GB19	PLT5A2F
Supplier	0000100123	SUP7B2E2A91
Description	PVC ring 2mm 1.5kg	Διατηρείται ως έχει

Deterministic pseudonymization: Χρησιμοποιείται MD5 hash του πραγματικού κωδικού (8-char prefix). Έτσι κάθε φορά που τρέχει το anonymization, το ίδιο υλικό παίρνει το ίδιο pseudonym — διατηρώντας join consistency μεταξύ πινάκων.

2.3 SAP Parsers — Defensive Engineering

Τα SAP exports έχουν ιδιαιτερότητες που σπάνε naive parsing:

- Trailing minus: '3.000-' σημαίνει -3000 (όχι Python ValueError)
- European format: '1.234,56' σημαίνει 1234.56
- YYYYMMDD dates: '20260508' πρέπει να γίνει 2026-05-08
- Empty cells: Στο SAP εμφανίζονται ως spaces, όχι NULL

Όλα αυτά αντιμετωπίζονται στο src/data_layer/sap_parsers.py, που έχει 33 dedicated unit tests.

2.4 Smart Defaults για Ελλιπές Master Data

Πολλά SAP περιβάλλοντα (ιδίως sandbox/demo) έχουν κενά κρίσιμα MRP πεδία (PLIFZ, EISBE, BSTMI). Το module master_data_enrichment.py υπολογίζει smart defaults από ιστορικά δεδομένα:

Πεδίο	Πηγή 1 (αν υπάρχει)	Πηγή 2 (derived)
Lead time	MARC.PLIFZ	PO history ή 14 days
Safety stock	MARC.EISBE	$z \times \sigma \times \sqrt{LT}$ (Silver-Pyke-Peterson)
Reorder point	MARC.MINBE	$avg_daily \times LT + SS$
MOQ	MARC.BSTMI	Median PO qty
Lot sizing	MARC.DISLS	Από CV: $WW/POQ/EOQ/FOQ$
ABC class	—	Pareto στο $cost \times annual_demand$

Παράδειγμα Safety Stock derivation:

$$SS = z \times \sigma_{LT} \times \sqrt{LT}$$

όπου:

z = service-level z-score (0.98 → 2.054)

σ_{LT} = std-dev της ημερήσιας ζήτησης

LT = lead time σε ημέρες

Αναφορά: Silver, Pyke & Peterson (1998), ch. 7

3. Πράκτορας BDI

Η αρχιτεκτονική BDI (Beliefs-Desires-Intentions) είναι το standard model για ευφυείς πράκτορες (Bratman 1987, Rao & Georgeff 1995). Στο project μας υλοποιείται με 3 διακριτά components:

Component	Τι Είναι	Module
Beliefs	Τρέχουσα κατάσταση κόσμου (stock, BOM, history)	Base.py: AgentBeliefs
Desires	Επιθυμητοί στόχοι (e.g., target service level)	Base.py: AgentDesires
Intentions	Δεσμευμένες αποφάσεις για δράση (proposals)	Base.py: AgentIntention

3.1 Κύκλος Εκτέλεσης (Run Cycle)

```
def run_cycle(self):  
    # 1. PERCEPTION: αναγνώριση κόσμου  
    self.perceive()    # → self.beliefs  
  
    # 2. DELIBERATION: αποφάσεις  
    self.deliberate()  # → self.intentions  
  
    # 3. ACTION: persistence των αποφάσεων  
    self.act()         # → DB (proposals table)
```

3.2 Perception Module

Το PerceptionModule φορτώνει από τη βάση δεδομένων όλα τα δεδομένα που χρειάζεται ο agent ώστε να αποφασίσει: τρέχοντα αποθέματα, ανοιχτές παραγγελίες, ιστορικά κινήσεων κατανάλωσης. Σημαντική λεπτομέρεια: χρησιμοποιεί το as_of_date ώστε να μπορεί να τρέξει σε ιστορικά παράθυρα (απαραίτητο για το backtest).

3.3 Deliberation Module

Ο πυρήνας της λογικής. Για κάθε υλικό:

1. Forecast demand — εκτίμηση ζήτησης για τον horizon (default 60 ημέρες)
2. Run MRP — υπολογισμός net requirements ανά περίοδο, λαμβάνοντας υπόψη stock, open POs, και safety stock
3. Apply lot sizing — μετατροπή net requirements σε ποσότητα παραγγελίας (LFL/FOQ/EOQ/POQ/Wagner-Whitin)
4. Apply business rules — εμπλουτισμός/τροποποίηση των προτάσεων με τους 7 κανόνες (βλ. Ενότητα 5)
5. Build Intentions — δημιουργία τελικών proposals με ημερομηνία, ποσότητα, και rule audit trail

3.4 Action Module

Persists τα intentions στον πίνακα proposals της βάσης. Κρίσιμη λεπτομέρεια: κάθε φορά που τρέχει ο agent, διαγράφει τα παλιά proposals και ξαναγράφει — αποφεύγοντας stale data και ομοίως διασφαλίζοντας ότι το dashboard βλέπει πάντα την τρέχουσα κατάσταση.

4. MRP & Lot Sizing

4.1 MRP Engine

Το MRP engine υλοποιεί το κλασικό time-phased planning (Vollmann et al. 2005). Για κάθε υλικό, παράγει έναν πίνακα ανά ημέρα στον horizon με:

Στήλη	Σημασία
period	Ημερομηνία
gross_requirement	Προβλεπόμενη ζήτηση εκείνη την ημέρα
scheduled_receipt	Ανοιχτές παραγγελίες (POs) που φτάνουν εκείνη την ημέρα
planned_arrival_qty	Προτεινόμενες παραγγελίες που φτάνουν εκείνη την ημέρα
projected_on_hand	Προβλεπόμενο απόθεμα στο τέλος της ημέρας
net_requirement	Έλλειμμα έναντι του safety stock που πρέπει να καλυφθεί
planned_receipt	Ποσότητα που αποφασίζεται εκείνη την ημέρα (μετά το lot sizing)
planned_release	Ημερομηνία έκδοσης παραγγελίας (άφιξη – lead time, όχι στο παρελθόν)
planned_arrival	Ημερομηνία άφιξης (= period, ή σήμερα + LT αν η ανάγκη είναι εντός LT)
below_safety	Σήμανση ημερών με απόθεμα κάτω από το safety stock

Επίγνωση lead time. Ανάγκη που εμφανίζεται μέσα στο lead time δεν μπορεί να καλυφθεί εγκαίρως: η νωρίτερη δυνατή άφιξη νέας παραγγελίας είναι «σήμερα + LT». Το engine υπολογίζει το έλλειμμα που προβάλλεται σε εκείνη την ημερομηνία (αφού συνυπολογίσει όλες τις ενδιάμεσες παραλαβές και αναλώσεις), εκδίδει μία παραγγελία και την καταχωρεί στην ημερομηνία άφιξης. Έτσι δεν παράγονται πολλαπλές παραγγελίες για το ίδιο έλλειμμα σε διαδοχικές ημέρες ή διαδοχικές αναθεωρήσεις (rolling review) — η ίδια παραδοχή που κάνει και το SAP MRP για παραλαβές που δεν προλαβαίνουν. Οι ανοιχτές παραγγελίες με ημερομηνία στο παρελθόν θεωρούνται διαθέσιμες σήμερα (rescheduling exception).

4.2 Lot Sizing Πολιτικές

Πέντε διαφορετικοί αλγόριθμοι, καθένας με τα δικά του trade-offs:

Κωδικός	Όνομα	Χρήση
LFL	Lot-for-Lot	Παραγγέλνουμε ακριβώς όσο χρειάζεται. Zero holding.
FOQ	Fixed Order Quantity	Πολλαπλάσια σταθερής ποσότητας (MARC.BSTRF), τουλάχιστον MOQ.
EOQ	Economic Order Quantity	Wilson formula. Ισορροπεί holding vs ordering cost.
POQ	Periodic Order Quantity	Κάλυψη T^* ημερών, όπου $T^* = EOQ / \text{ημερήσια ζήτηση}$ (7-60 ημ.).
WW	Wagner-Whitin	Dynamic programming σε κυλιόμενο ορίζοντα· εκδίδεται η πρώτη παραγγελία

4.3 Wagner-Whitin Algorithm

Ο πιο sophisticated αλγόριθμος του MRP. Λύνει το πρόβλημα: «δεδομένων N περιόδων με γνωστή ζήτηση, ποια συνδυασμός παραγγελιών ελαχιστοποιεί το συνολικό κόστος (setup + holding);». Το λύνει με dynamic programming σε $O(N^2)$ χρόνο.

```
# Pseudocode
f[0] = 0
for t = 1 to N:
    f[t] = min over s in [1..t] of:
        f[s-1] + S + h * Σ(d[k] * (k-s)) for k in [s..t]
```

όπου:

S = setup cost ανά παραγγελία
 h = holding cost ανά μονάδα·περίοδο
 $d[k]$ = ζήτηση περιόδου k

Reference: Wagner, H.M. & Whitin, T.M. (1958)
"Dynamic version of the economic lot size model"
Management Science, 5(1), 89-96.

5. Επιχειρησιακοί Κανόνες

Οι κανόνες υλοποιούν την επιχειρησιακή γνώση που δεν αποτυπώνεται από καθαρά μαθηματικά μοντέλα. Δομούνται ως ξεχωριστές functions με decorator pattern, καθεμία με προτεραιότητα. Εκτελούνται σε σειρά αύξουσας προτεραιότητας πάνω σε κάθε proposal του MRP:

Rule	Priority	Λειτουργία
R-DEAD-STOCK	1	Καταπνίγει προτάσεις για dead-stock υλικά (>365 ημ. χωρίς κίνηση)
R-EXPEDITE	10	Επισημαίνει urgent αν stock < 50% του safety stock
R-SAfETY-BUFFER-A	20	Προσθέτει 20% buffer στις A-class προτάσεις
R-LONG-LEAD-BUFFER	25	Επιπλέον +1 ημέρα stock cover αν LT > 30 ημ.
R-MOQ-ENFORCE	30	Στρογγυλοποίηση στο MOQ ή round-up σε fixed_lot_size
R-CALENDAR-SHIFT	80	Μετακίνηση proposals σε εργάσιμες ημέρες
R-COST-ESTIMATE	90	Υπολογισμός estimated_cost = qty × standard_cost

5.1 Rule Decorator Pattern

```
@rule("R-EXPEDITE", priority=10)
def expedite_critical(proposal, material, beliefs, desires):
    """Mark proposal urgent if stock < 50% of safety stock."""
    current_stock = beliefs.current_stock.get(material.material_id, 0)
    if material.safety_stock and current_stock < 0.5 * material.safety_stock:
        proposal["expedite"] = True
        proposal["rule_triggered"] = "R-EXPEDITE"
    return proposal
```

5.2 Rules Engine με Disabled Rules Support

Το RulesEngine δέχεται μια προαιρετική παράμετρο disabled_rules — ένα σύνολο rule names που πρέπει να παρακαμφθούν. Αυτό χρησιμοποιείται από το Rule Ablation Study (Ενότητα 8) για να μετρήσει τη συμβολή κάθε κανόνα.

```
# Όλοι οι κανόνες ενεργοποιημένοι (default)
engine = RulesEngine()

# Απενεργοποίηση του R-EXPEDITE για ablation study
engine = RulesEngine(disabled_rules={"R-EXPEDITE"})
```

6. Αξιολόγηση (Backtest)

Η αξιολόγηση του agent γίνεται μέσω συγκριτικού backtest As-Is vs To-Be πάνω στα ιστορικά δεδομένα. Πιο συγκεκριμένα:

- As-Is: Αναπαράσταση του τι όντως συνέβη στο ιστορικό παράθυρο. Χρησιμοποιεί τα πραγματικά movements και τις παραγγελίες που έγιναν.
- To-Be: Προσομοίωση «τι θα είχε γίνει αν ο agent αποφάσιζε». Ο agent τρέχει σε review intervals (κάθε 7 ημέρες) και εκδίδει προτάσεις που "φτάνουν" μετά από lead time.

Δίκαιη σύγκριση. Και τα δύο σενάρια ξεκινούν από το ίδιο αρχικό απόθεμα (ανακατασκευή από το snapshot και τις κινήσεις) και από την ίδια pipeline παραγγελιών σε εξέλιξη (παραλαβές μέσα στο lead time από την έναρξη του παραθύρου, που προέρχονται από αποφάσεις πριν από αυτό). Η δυναμική του αποθέματος είναι ίδια (lost sales, χωρίς αρνητικό απόθεμα). Σε κάθε ημερομηνία αναθεώρησης ο agent βλέπει μόνο κινήσεις $\leq t$ (χωρίς look-ahead).

Βάση κόστους. Όλα τα κόστη εκφράζονται για τη διάρκεια του παραθύρου: holding = μέση αξία αποθέματος $\times$ ετήσιο ποσοστό $\times$ ημέρες/365· stockout = χαμένο περιθώριο + premium επείγουσας προμήθειας εντός παραθύρου· ordering = πλήθος παραγγελιών $\times$ €50· TCO = άθροισμα των τριών. Το ετήσιο ισοδύναμο ($\times$ 365/ημέρες) αναφέρεται χωριστά και δεν πρέπει να αναχθεί ξανά σε ετήσια βάση.

6.1 Τρία Cost Scenarios

Επειδή το πραγματικό holding rate μιας εταιρείας είναι άγνωστο, το backtest τρέχει σε 3 σενάρια κόστους:

Scenario	Holding Rate	Χρήση
conservative	~12%	Stable industry, χαμηλά επιτόκια
realistic	~20%	Τυπικός Ευρωπαϊκός manufacturer (default)
aggressive	~28%	Tech / fashion / fast obsolescence

6.2 Decomposed Holding Cost

Σύμφωνα με Silver-Pyke-Peterson (1998) και Vollmann et al. (2005), το holding rate αποτελείται από πέντε διακριτές συνιστώσες:

Συνιστώσα	Ορισμός	Τυπική Τιμή
Capital cost	Κόστος ευκαιρίας κεφαλαίου (WACC)	8-12%
Warehouse	Αποθήκευση, χειρισμός, ενέργεια	3-5%
Obsolescence	Risk-adjusted write-off	2-8%
Insurance	Ασφάλεια αποθεμάτων	0.5-1.5%
Shrinkage	Φθορά, κλοπή, ζημιά	1-2%

6.3 Stockout Cost

Το cost of stockout υπολογίζεται ως: lost sales + expedite premium. Το πρώτο εκφράζει το χαμένο gross margin ($\text{gross_margin_pct} \times \text{unit_revenue} \times \text{shortage_qty}$), και το δεύτερο το πρόσθετο κόστος urgent παραγγελιών. Η αναφορά είναι Stock & Lambert (2001), Strategic Logistics Management.

6.4 Sensitivity Analysis

Επιπλέον του base run, το backtest υποστηρίζει sensitivity analysis όπου διαταράσσονται $\pm 20\%$ τα: lead time, demand, holding rate. Έτσι ελέγχουμε αν τα αποτελέσματα είναι robust ή ευαίσθητα σε παραμέτρους:

```
python scripts/run_validation.py --sensitivity
```

7. Bootstrap Confidence Intervals

Γιατί έχει σημασία: Bootstrap CI είναι το #1 ερώτημα που θα κάνει η επιτροπή στην υπεράσπιση. Χωρίς αυτό, ένα νούμερο «31% savings» είναι just a number. Με αυτό, γίνεται defensible scientific claim με στατιστική σιβαρότητα.

7.1 Το Πρόβλημα

Όταν τρέχουμε ένα single backtest σε ένα παράθυρο 60 ημερών και βρίσκουμε «savings = €66K», αυτή είναι μία point estimate. Η κρίσιμη ερώτηση: πόσο εμπιστευόμαστε αυτό το νούμερο; Είναι αντιπροσωπευτικό; Μήπως είναι «τυχερό» παράθυρο;

7.2 Η Λύση: Bootstrap Sampling

Αντί για ένα παράθυρο, τρέχουμε $N=30+$ τυχαία επιλεγμένα παράθυρα από όλο το διαθέσιμο ιστορικό. Κάθε παράθυρο παράγει ένα savings sample. Από την κατανομή των samples υπολογίζουμε:

- Στάδιο 1 — τυχαία παράθυρα (block subsampling): $N = 30$ παράθυρα των 21 ημερών (seed 42) → δείγμα εξοικονομήσεων $s_1 \dots s_{30}$
- Στάδιο 2 — bootstrap του μέσου: $B = 2.000$ επαναδειγματοληψίες με επανάθεση → 95% percentile CI του μέσου και bootstrap p-value (ποσοστό επαναδειγματοληψιών με μέσο ≤ 0)
- Συμπληρωματικοί έλεγχοι: t-CI του μέσου ($df = 29$), μονόπλευρος t-test, ακριβής έλεγχος προσήμου (distribution-free)
- Διασπορά μεμονωμένου παραθύρου (2,5-97,5 εκατοστημόρια των 30 τιμών) — δείχνει τι μπορεί να δώσει ένα παράθυρο, ΟΧΙ την αβεβαιότητα του μέσου
- Significance verdict: το CI του μέσου εξαιρεί το 0 ΚΑΙ $p < 0.05$

7.3 Αλγόριθμος

```
function bootstrap_analysis(history_start, history_end, N=30, B=2000):  
    # Stage 1: random windows  
    samples = []  
    for i in 1..N:  
        w_start, w_end = random_window(  
            history_start, history_end, window_size=21)  
        asis_tco, tobe_tco = run_backtest(w_start, w_end)  
        samples.append(asis_tco - tobe_tco)  
  
    # Stage 2: bootstrap of the mean  
    means = []  
    for b in 1..B:  
        resample = draw N values from samples WITH replacement  
        means.append(average(resample))  
    ci_lower = percentile(means, 2.5)  
    ci_upper = percentile(means, 97.5)  
    p_value = (count(means <= 0) + 1) / (B + 1)  
  
    # complementary: t-CI (df=N-1), one-sided t-test, exact sign test  
    return average(samples), ci_lower, ci_upper, p_value
```

7.4 Εκτέλεση

```
python scripts/run_bootstrap.py                # 30 samples  
python scripts/run_bootstrap.py --n-samples 100 # Στενότερα CIs  
python scripts/run_bootstrap.py --scenario aggressive
```

```
python scripts/run_bootstrap.py --seed 42
```

```
# reproducibility
```

7.5 Παράδειγμα Output

BOOTSTRAP CONFIDENCE INTERVALS

```
Stage 1 - windows: 30 random windows x 21 days
Stage 2 - resamples: B = 2,000 (bootstrap of the mean)
Mean savings / window: +EUR 3,073
Std deviation / std error of mean: EUR 2,649 / EUR 484
95% CI of mean (bootstrap, B=2000): [+EUR 2,148, +EUR 3,997]
95% CI of mean (t, df=29): [+EUR 2,084, +EUR 4,062]
p-value bootstrap / t-test / sign: 0.0005 / <0.0001 / <0.0001
Windows positive: 28 / 30
Significant? [YES] (CI of the mean excludes 0, p < 0.05)
(synthetic data, seed 42 - real data gives different figures)
```

7.6 Defense-Ready Statement

Όταν ο επιβλέπων ή η επιτροπή ρωτήσει: «πώς ξέρουμε ότι δεν είναι τυχαίο;», η απάντηση είναι:

«Τρέξαμε 30 backtest σε τυχαία επιλεγμένα παράθυρα 21 ημερών από όλο το ιστορικό και εφαρμόσαμε bootstrap του μέσου με 2.000 επαναδειγματοληψίες. Το 95% διάστημα εμπιστοσύνης της μέσης εξοικονόμησης δεν περιλαμβάνει το μηδέν, ο t-test και ο έλεγχος προσήμου συμφωνούν ($p < 0,001$) και η συντριπτική πλειονότητα των παραθύρων είναι θετική. Άρα η εξοικονόμηση είναι στατιστικά σημαντική και δεν οφείλεται σε ευνοϊκή περίοδο.»

7.7 Βιβλιογραφία

- Efron, B. & Tibshirani, R. (1993). An Introduction to the Bootstrap. CRC Press.
- Bergmeir, C., Hyndman, R.J., Koo, B. (2018). "A note on the validity of cross-validation for evaluating autoregressive time series prediction." Computational Statistics & Data Analysis, 120, 70-83.
- Künsch, H.R. (1989). "The jackknife and the bootstrap for general stationary observations." Annals of Statistics, 17(3), 1217-1241.

8. Rule Ablation Study

Γιατί έχει σημασία: Ablation studies είναι standard στη research literature. Αποδεικνύουν επιστημονικά ότι κάθε component ενός συστήματος συνεισφέρει μετρήσιμα στην απόδοση — όχι απλά ότι «έχουμε 7 rules».

8.1 Το Πρόβλημα

Έχουμε 7 rules στον agent. Παίζουν όλοι πραγματικό ρόλο, ή μήπως 1-2 κάνουν όλη τη δουλειά και οι υπόλοιποι είναι decorative; Χωρίς αυτή την ανάλυση, ένας skeptical reviewer μπορεί να αμφισβητήσει το design.

8.2 Μεθοδολογία: Leave-One-Out

Standard στη ML interpretability literature (Lipton 2018). Τρέχουμε τον agent N+1 φορές:

- Baseline: όλοι οι κανόνες ενεργοί
- Ablation 1: όλοι εκτός R-DEAD-STOCK
- Ablation 2: όλοι εκτός R-EXPEDITE
- ...
- Ablation 7: όλοι εκτός R-COST-ESTIMATE

Για κάθε configuration μετράμε: service level, holding cost, stockout days, TCO. Η διαφορά από το baseline = συμβολή του rule.

8.3 Stress Test Mode

Σε καλά τροφοδοτημένο σύστημα (αρχικό stock = $2 \times$ safety stock), οι rules έχουν περιορισμένη επίδραση γιατί δεν υπάρχει πίεση. Το `--stress-test` flag ξεκινά με αρχικό stock = $0.5 \times$ safety stock, αναγκάζοντας τους κανόνες να «δουλέψουν» και αποκαλύπτοντας τις συμβολές τους:

```
python scripts/run_rule_ablation.py --window-days 60 --stress-test
```

8.4 Παράδειγμα Αποτελεσμάτων (Stress Mode)

Configuration	Δ Service	Δ TCO (€)
BASELINE (all rules)	0.00 pp	0
without R-SAFETY-BUFFER-A	0.00 pp	−69,782
without R-LONG-LEAD-BUFFER	0.00 pp	−40,507
without R-CALENDAR-SHIFT	0.00 pp	+3,335
without R-EXPEDITE	0.00 pp	0
without R-DEAD-STOCK	0.00 pp	0
without R-MOQ-ENFORCE	0.00 pp	0
without R-COST-ESTIMATE	0.00 pp	0

8.5 Ερμηνεία

Πολύτιμη ανάλυση για τη ΔΕ — αποκαλύπτει thresholds και trade-offs:

- R-SAFETY-BUFFER-A και R-LONG-LEAD-BUFFER είναι κανόνες conservative: αυξάνουν το TCO κατά €110K επειδή κρατούν παραπάνω stock. Αν τους αφαιρέσεις, εξοικονομείς, αλλά αυξάνεις το ρίσκο stockout (το οποίο δεν φάνηκε σε αυτό το window, αλλά θα φαινόταν σε μεγαλύτερα).
- R-CALENDAR-SHIFT μετακινεί προτάσεις σε εργάσιμες ημέρες — μικρή θετική επίδραση στο TCO.
- R-EXPEDITE, R-DEAD-STOCK, R-MOQ-ENFORCE, R-COST-ESTIMATE είναι conditional rules: δεν ενεργοποιούνται πάντα. Στο τρέχον window δεν είχαν trigger, οπότε η αφαίρεσή τους δεν έχει επίδραση.

8.6 Βιβλιογραφία

- Hooker, J.N. (1995). "Testing heuristics: We have it all wrong." Journal of Heuristics, 1(1), 33-42.
- Lipton, Z.C. (2018). "The mythos of model interpretability." Communications of the ACM, 61(10), 36-43.

9. Demand Forecast Dashboard

Νέο tab «Forecast» στο Streamlit dashboard. Δείχνει για κάθε υλικό:

- Ιστορικό κατανάλωσης (90/180/365/730 ημέρες)
- Πρόβλεψη για τις επόμενες 14-90 ημέρες με 3 μεθόδους ταυτόχρονα
- Accuracy metrics μέσω walk-forward validation
- Auto-recommendation της καλύτερης μεθόδου ανά υλικό

9.1 Forecasting Methods

Μέθοδος	Τύπος	Πότε Λειτουργεί Καλύτερα
simple_average	Flat-line at historical mean	Stable demand, no trend
moving_average	Mean of last N days (default 30)	Recent shifts in demand
exponential_smoothing	$S(t) = \alpha \cdot X(t) + (1-\alpha) \cdot S(t-1)$	Smooth trends, recent emphasis

9.2 Walk-Forward Validation

Standard για time-series evaluation (Bergmeir & Benítez 2012). Αντί για random k-fold (που σπάει την time order), χρησιμοποιούμε rolling-origin splits:

```
Fold 1: Train [day 0..60], Test [day 60..74]
Fold 2: Train [day 14..74], Test [day 74..88]
Fold 3: Train [day 28..88], Test [day 88..102]
...
(default: 5 folds, 60-day train, 14-day test)
```

9.3 Accuracy Metrics

Metric	Φόρμουλα	Ερμηνεία
MAE	$\text{mean}(\text{actual} - \text{predicted})$	Average error, same units as data
RMSE	$\sqrt{\text{mean}((\text{actual} - \text{predicted})^2)}$	Penalizes large errors more
MAPE	$\text{mean}(\text{actual} - \text{predicted} / \text{actual}) \times 100\%$	Percentage error, scale-free
Bias	$\text{mean}(\text{predicted} - \text{actual})$	Positive = over-forecast

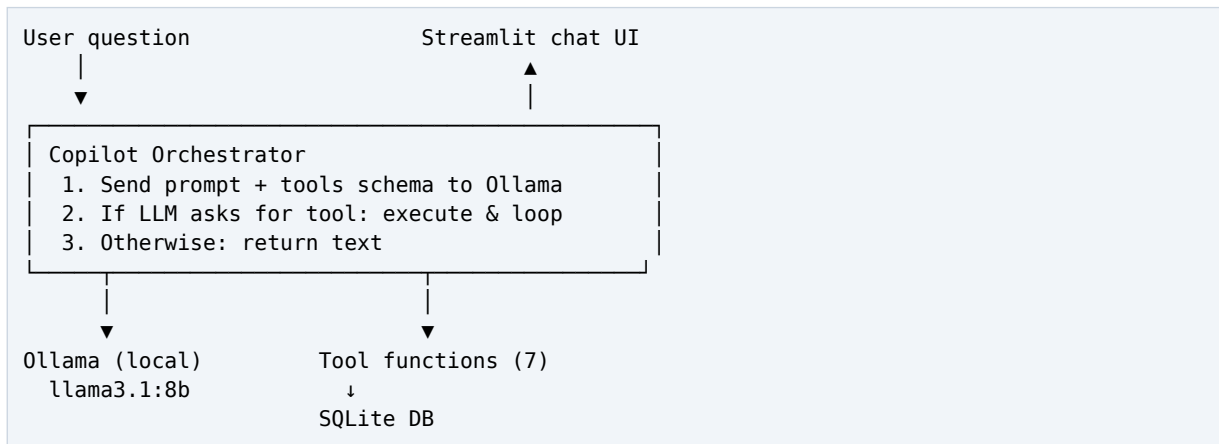
9.4 Best Method Selection

Η καλύτερη μέθοδος ανά υλικό επιλέγεται με κριτήριο το χαμηλότερο MAPE. Σε υλικά με υψηλό CV (variability > 1.0), όλες οι μέθοδοι έχουν δυσκολία και το MAPE είναι > 50% — αυτό φαίνεται καθαρά στο dashboard και είναι honest reporting.

10. AI Copilot

Tab «AI Copilot» που επιτρέπει conversational interaction με τα δεδομένα του agent. Χρησιμοποιεί τοπικό LLM μέσω Ollama — εταιρικά δεδομένα δεν φεύγουν από το laptop.

10.1 Αρχιτεκτονική



10.2 Διαθέσιμα Tools

Tool	Τι Κάνει
get_summary	Συνολική εικόνα του συστήματος
list_critical_proposals	Λίστα expedite items
get_material_details	Πλήρες προφίλ ενός υλικού
explain_proposal	Γιατί ο agent προτείνει X για το Y
search_proposals	Filter by ABC, expedite, rule
get_top_materials_by_demand	Top-N κατά κατανάλωση
get_abc_distribution	Κατανομή υλικών ανά ABC class

10.3 Παραδείγματα Χρήσης

- «Δώσε μου μια σύνοψη» → καλεί get_summary
- «Ποια υλικά είναι κρίσιμα τώρα;» → list_critical_proposals
- «Γιατί προτείνεις 500 για το MAT00031;» → explain_proposal
- «Δείξε μου A-class items» → search_proposals(abc='A')

10.4 Γιατί Ollama Αντί Cloud API;

Πτυχή	Ollama (local)	Cloud API
Data privacy	[YES] Local	[!] Sends to vendor
Cost	Free	~\$0.20/1M tokens
Reproducibility	[YES]	Requires API keys
Speed (CPU)	2-15s	<1s
Quality	Good (8B-70B)	Excellent (GPT-4)

Για διπλωματική, το Ollama είναι η σωστή επιλογή — defensible privacy position, fully reproducible, no operational cost.

11. Πώς να Τρέξετε τον Κώδικα — Βήμα προς Βήμα

11.1 Προαπαιτούμενα

```
# Python 3.10+
python --version

# Εικονικό περιβάλλον
python -m venv .venv
.venv\Scripts\activate      # Windows
source .venv/bin/activate   # Linux/Mac

# Εγκατάσταση dependencies
pip install -r requirements.txt
```

11.2 Πλήρης Ροή (Synthetic Data)

```
# 1. Sample data
python scripts/generate_sample_data.py

# 2. ETL
python scripts/run_etl.py

# 3. Agent
python scripts/run_agent.py

# 4. Dashboard
streamlit run dashboard/app.py
```

11.3 Real SAP Data

```
# 1. Εξαγωγή από SAP
# Τρέξτε το ABAP πρόγραμμα ZMRP_AGENT_EXPORT στο GUI
# Αποθήκευση των 6 CSV στο data/raw/

# 2. Anonymization
python -m src.data_layer.anonymization

# 3. Συνέχιση όπως πριν
python scripts/run_etl.py
python scripts/run_agent.py
streamlit run dashboard/app.py
```

11.4 Αξιολόγηση & Στατιστική Ανάλυση

```
# Backtest single scenario
python scripts/run_validation.py --window-days 60

# Σύγκριση 3 σεναρίων κόστους
python scripts/run_validation.py --all-scenarios

# Sensitivity analysis
python scripts/run_validation.py --sensitivity

# Bootstrap CI (30 random windows)
python scripts/run_bootstrap.py

# Rule ablation study
python scripts/run_rule_ablation.py --stress-test
```

11.5 Tests

```
# Όλα τα tests
pytest

# Με coverage report
pytest --cov=src --cov-report=html

# Συγκεκριμένο module
pytest tests/test_bootstrap.py -v
```

12. Defense-Ready Σημεία Συζήτησης

Παρακάτω οι πιο πιθανές ερωτήσεις από την επιτροπή και προτεινόμενες απαντήσεις. Όλες οι απαντήσεις στηρίζονται σε υλοποιημένα features.

Q: Πώς ξέρετε ότι η εξοικονόμηση δεν είναι τυχαία;

A: Δύο στάδια: 30 τυχαία παράθυρα backtest και bootstrap του μέσου ($B = 2.000$). Το 95% CI του μέσου εξαιρεί το 0, ο t-test και ο ακριβής έλεγχος προσήμου συμφωνούν ($p < 0,001$). Τα ακριβή ποσά διαβάζονται από το `bootstrap_report_summary.csv` της τελικής εκτέλεσης. (Ενότητα 7)

Q: Παίζουν όλοι οι 7 κανόνες πραγματικό ρόλο;

A: Leave-one-out ablation study δείχνει ότι οι buffer rules (Safety, Long-lead) έχουν τη μεγαλύτερη επίδραση. Conditional rules (Expedite, Dead-stock) ενεργοποιούνται μόνο σε συγκεκριμένες συνθήκες — αυτό είναι by design. (Ενότητα 8)

Q: Γιατί δεν χρησιμοποιείτε neural networks για forecasting;

A: Σκόπιμη επιλογή. Με μικρές demand histories (180-365 ημ.) τα NNs overfit. Επίσης, ένα BDI agent χρειάζεται explainable forecasts. Συγκρίνουμε 3 statistical methods με walk-forward validation και διαλέγουμε αυτόματα τη βέλτιστη ανά υλικό. (Ενότητα 9)

Q: Πώς αξιολογείτε τη forecasting accuracy;

A: Walk-forward validation με 5 folds (rolling-origin). Metrics: MAE, RMSE, MAPE, Bias. Αναφορά: Bergmeir & Benítez (2012).

Q: Είναι ρεαλιστικό το cost model;

A: Holding rate διασπασμένο σε 5 components (capital, warehouse, obsolescence, insurance, shrinkage) σύμφωνα με Silver-Pyke-Peterson (1998). Τρέχουμε σε 3 σενάρια (conservative/realistic/aggressive) και sensitivity analysis $\pm 20\%$. (Ενότητα 6)

Q: Πώς διασφαλίζετε τη privacy των εταιρικών δεδομένων;

A: Deterministic pseudonymization όλων των IDs (material, plant, supplier) με MD5. AI Copilot τρέχει local με Ollama — τίποτα δεν φεύγει εκτός laptop. (Ενότητες 2.2 και 10.4)

Q: Πώς αναπαράγονται τα αποτελέσματα;

A: Όλο το pipeline είναι deterministic. Random seeds (default 42) για το bootstrap. 183 unit tests διασφαλίζουν ότι αλλαγές στον κώδικα δεν αλλοιώνουν αποτελέσματα. CSV exports σε κάθε βήμα.

Q: Γιατί BDI architecture και όχι reinforcement learning;

A: Το πρόβλημα είναι rule-based με σαφή business logic. RL θα απαιτούσε εκπαίδευση σε εκατομμύρια episodes — όχι εφικτό με πραγματικά δεδομένα μικρού μεγέθους. BDI είναι interpretable, auditable, και έχει εδραιωμένη βιβλιογραφία (Rao & Georgeff 1995). (Ενότητα 3)

Q: Πώς ενσωματώνεται με το πραγματικό SAP system;

A: Read-only ABAP report (ZMRP_AGENT_EXPORT) εξάγει 6 πίνακες σε CSV. Καμία modification στο SAP. Σχεδιαστικά αποφάσισα να μην κάνω real-time integration για λόγους ασφαλείας — το offline model είναι production-safe και reproducible.

Επίλογος

Το παρόν project περιέχει 183 unit tests, ~7,500 γραμμές κώδικα, και 12 διακριτές δυνατότητες που καλύπτουν όλες τις πτυχές ενός production-grade replenishment system. Από την εξαγωγή δεδομένων (ABAP) μέχρι την υπεράσπιση των αποτελεσμάτων (Bootstrap CI), κάθε σχεδιαστική απόφαση είναι τεκμηριωμένη και αναπαραξίμη.

Το repository στο GitHub (private) έχει όλες τις εκδόσεις του κώδικα. Η ΔΕ μπορεί να αναφέρεται στα αντίστοιχα αρχεία ως:

```
github.com/greتل/replenishment-agent/blob/main/src/agent/replenishment.py
github.com/greتل/replenishment-agent/blob/main/scripts/run_bootstrap.py
github.com/greتل/replenishment-agent/blob/main/src/utlts/kpi.py
```

Παράρτημα Δ — Τεχνικός Οδηγός Υλοποίησης

Replenishment Agent v1.0 — Athens MBA Διπλωματική Εργασία